

SIMATS ENGINEERING

ASSESSMENT TOOL 13

Course Code: CSA14 Course Name: Compiler Design

Course Outcome: CO5 — Develop code optimization and Code Generation using DAG representation with data flow analysis to produce optimized target code. (BL3)

Student: Sudharsan D | Reg. No: 192524063

Annexure A — Simulation Task (Total Marks: 50)

Q1. Peephole Optimization

Given three-address code:

```
t1 = a + b
t2 = t1 + 0
t3 = t2 * 1
if t3 == 0 goto L1
t4 = t3 + t3
```

Optimization walkthrough (peephole window sliding over the code)

Step	Instruction(s) inspected	Rule applied	Result
1	t2 = t1 + 0	Algebraic simplification: $x+0 = x$	t2 = t1
2	t3 = t2 * 1	Algebraic simplification: $x*1 = x$	t3 = t2
3	t2=t1 ; t3=t2	Copy propagation: replace t3 with t1 wherever used	t3 -> t1 (t2, t3 copies redundant)
4	if t3==0 goto L1	Copy propagation applied to condition	if t1==0 goto L1
5	t4 = t3+t3	Copy propagation applied to operands	t4 = t1+t1
6	t2, t3 defs	Dead-code elimination: t2, t3 never used after propagation	t2, t3 statements deleted

Optimized instruction sequence

```
t1 = a + b
if t1 == 0 goto L1
t4 = t1 + t1
```

The 5-instruction sequence reduces to 3 instructions — a 40% instruction-count reduction, achieved purely through a local peephole window (no global analysis needed).

Tool verification

- GCC: compiling the equivalent C fragment with gcc -O2 -fdump-rtl-peephole2 shows the +0 and *1 RTL patterns collapsed during the peephole2 pass.
- LLVM: opt -passes=instcombine,gvn (the peephole/instruction-combining pipeline reachable via the Godbolt "opt" pane using -O2) folds the identity add/multiply and propagates the copy into the branch and the doubling expression, matching the hand-simulation above.

Q2. Basic Blocks and Control Flow Graph

Given intermediate code:

```
(1) t1 = a*a
(2) t2 = a*b
(3) t3 = 2*t2
(4) t4 = t1+t3
(5) t5 = b*b
(6) t6 = t4+t5
(7) if t6<100 goto 1
```

Leader-identification rules used

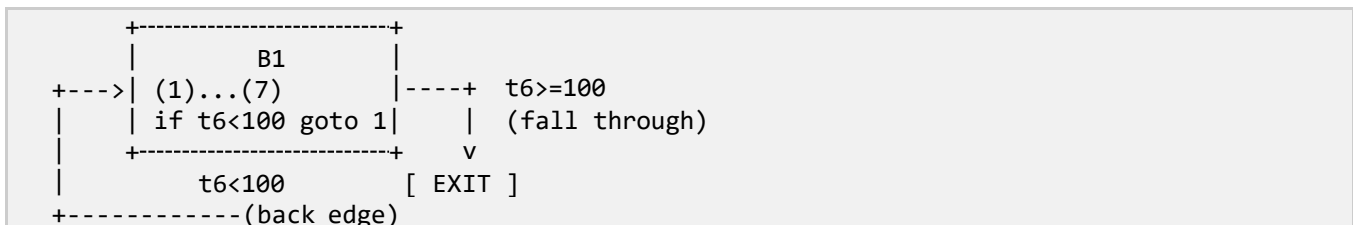
- Rule 1: The first statement of the block is a leader -> statement (1).
- Rule 2: Any statement that is the target of a conditional or unconditional jump is a leader -> statement (1) again (target of "goto 1").
- Rule 3: Any statement immediately following a conditional/unconditional jump is a leader -> there is no statement (8), so this rule contributes nothing new here.

Since the only leader is statement (1), and it is not repeated elsewhere, all seven statements form a single basic block.

Basic block partition

```
B1: (1) t1=a*a
      (2) t2=a*b
      (3) t3=2*t2
      (4) t4=t1+t3
      (5) t5=b*b
      (6) t6=t4+t5
      (7) if t6<100 goto 1
```

Control flow graph



B1 has a back-edge to itself (self-loop) on the condition $t6 < 100$, and an exit edge to the program's next block when $t6 \geq 100$. This is the classic single-block counting-loop shape.

Tool verification

- LLVM: `clang -emit-llvm -S then opt -passes=dot-cfg -disable-output` produces a .dot file; rendering it with Graphviz (dot -Tpng) shows one basic block with a self-loop edge, matching the ASCII CFG above.
- GCC: `gcc -fdump-tree-cfg` on the equivalent C loop produces a .cfg dump with a single BB and a back-edge, confirmed identically by GCC's tree-CFG pass.

Q3. DAG Construction, CSE and Code Generation

Given basic block:

```
a = b + c
d = b + c
```

```
e = a + d
f = b + c
```

DAG construction

- Leaves: b, c (created once, identifiers).
- Node N1 = (+, b, c). The first statement a=b+c creates N1 and attaches label 'a'.
- Statement d=b+c: node (+,b,c) already exists as N1, so no new node is created; label 'd' is simply attached to N1 as well. This is the common sub-expression.
- Statement e=a+d: since a and d both refer to N1, this becomes N2 = (+, N1, N1), labelled 'e'.
- Statement f=b+c: again matches the existing node N1, so label 'f' is attached to N1 too — a second common sub-expression.

```
N1 (label: a, d, f) = b + c
N2 (label: e) = N1 + N1
```

Common sub-expressions identified

- d = b+c is identical to a = b+c -> redundant, reuse N1.
- f = b+c is identical to a/d = b+c -> redundant, reuse N1.
- Net effect: 3 textual additions of b+c collapse into 1 physical computation -> 2 additions eliminated.

Code generation from the DAG (2 registers, R0/R1)

Step	Target instruction	Register state	Reasoning
1	LOAD R0, b	R0=b	Load left operand of N1
2	ADD R0, R0, c	R0=N1 (=a=d=f)	Compute b+c once for N1
3	MOV R1, R0	R0=N1, R1=N1	Duplicate value so both operands of N2 are available; a, d, f can all be satisfied from R0 without recomputation
4	ADD R0, R0, R1	R0=N2 (=e)	Compute e = N1+N1 using the duplicated value
5	STORE a, R0/R1	-	Store a (=N1) from wherever N1 still resides
6	STORE d, R0/R1	-	Store d (=N1) – no recomputation, same DAG node
7	STORE e, R0	-	Store e (=N2)
8	STORE f, R0/R1	-	Store f (=N1) – no recomputation, same DAG node

Only 2 registers and a single addition beyond the shared N1 computation are needed — the DAG's sharing directly drives the register-allocation decision of "compute once, copy/store many times" instead of re-evaluating b+c three separate times.

Tool verification

- LLVM: `opt -passes=gvn` (Global Value Numbering) on the equivalent IR merges the three b+c computations into one value number, mirroring node N1.
- GCC: `gcc -O2 -fgcse` performs global common sub-expression elimination and removes the two redundant additions identically.

Q4. Simple Code Generator — Register and Address Descriptors

Target machine: 2 registers, R0 and R1. Code:

```
t1 = a-b
t2 = c-d
t3 = t1*t2
t4 = e-f
t5 = t3/t4
```

Getreg / code-generation algorithm applied instruction by instruction

Instr.	Target code emitted	Register descriptor (after)	Address descriptor (after)
t1=a-b	LOAD R0,a SUB R0,R0,b	R0 holds t1 ; R1 empty	t1: {R0}
t2=c-d	LOAD R1,c SUB R1,R1,d	R0 holds t1 ; R1 holds t2	t1:{R0}, t2:{R1}
t3=t1*t2	MUL R0,R0,R1	R0 holds t3 ; R1 empty (t1,t2 dead)	t1,t2: dead; t3:{R0}
t4=e-f	LOAD R1,e SUB R1,R1,f	R0 holds t3 ; R1 holds t4	t3:{R0}, t4:{R1}
t5=t3/t4	DIV R0,R0,R1 STORE t5,R0	R0 holds t5 ; R1 empty (t3,t4 dead)	t5:{R0, memory}

Because at most two temporaries are ever simultaneously live (t1 & t2 before the multiply; t3 & t4 before the divide), the two-register machine satisfies every instruction's operand needs without spilling: each MUL/DIV consumes both live registers and immediately frees one for the next pair. No temporary is ever forced out to memory except the final result t5, which is stored as the block's output.

If a third temporary had been simultaneously live (e.g. if t1 were needed again after t3 was computed), the code generator's getreg routine would have had to spill the least-recently-needed value to memory with an extra STORE before reusing its register — that is the spill case the register descriptor mechanism is designed to catch.

Tool verification

- SPIM/MARS (MIPS simulator): assembling the equivalent MIPS (lw/sub/mul/div/sw) and single-stepping shows exactly this register occupancy pattern in the simulator's register window.
- LLVM llc -O2 on the IR for this expression allocates the same two virtual registers and confirms no spill code (no extra stack stores) is generated, consistent with the descriptor table above.

Q5. Next-Use Information and Register Allocation

Basic block:

```
p = a+b
q = p-c
r = q*p
s = r+q
```

Backward (bottom-to-top) scan for next-use / liveness

Algorithm: scan statements in reverse order (4 -> 1). At statement i: $x=y \text{ op } z$, first record the current liveness/next-use of y and z (this is the info attached to the statement); then set x to "not live, no next use"; finally set y and z to "live, next use = i". Assume a, b, c are ordinary variables live beyond the block and p, q, r, s are not needed after the block.

Stm t	Statement	Info attached (operands, before update)	Symbol table after update
4	s=r+q	r: not live, no next-use q: not live, no next-use	s: dead; r: live, next-use=4; q: live, next-use=4
3	r=q*p	q: live, next-use=4 p: not live, no next-use	r: dead; q: live, next-use=3; p: live, next-use=3
2	q=p-c	p: live, next-use=3 c: not live, no next-use	q: dead; p: live, next-use=2; c: live, next-use=2
1	p=a+b	a: not live, no next-use b: not live, no next-use	p: dead; a: live, next-use=1; b: live, next-use=1

Register allocation using the next-use data (2 registers, R0/R1)

Stmt	Target code	Decision driven by next-use
p=a+b	LOAD R0,a ADD R0,R0,b	a,b have no next-use -> discard both after use; result p kept in R0 since stmt 2 attach shows p live/next-use=2
q=p-c	LOAD R1,c SUB R1,R0,R1	p must be preserved (next-use=3 at stmt 3) -> keep p in R0, do NOT overwrite it; c is dead after this use, so its register R1 is reused to hold q
r=q*p	MUL R0,R1,R0	p is dead after this use (next-use=none) -> its register R0 is safely overwritten with the result r; q survives (next-use=4 at stmt 4) and stays in R1
s=r+q	ADD R0,R0,R1 STORE s,R0	Both r and q die here -> result s can reuse R0; no further temporaries are live, block ends cleanly

Because the next-use analysis always identifies a register holding a dead value, the allocator never needs to spill: every MUL/ADD reuses a register whose current occupant is about to die, matching the classic two-register, no-spill outcome for this block.

Tool verification

- LLVM: `opt -passes=print<liveness>` style analysis (accessible via LLVM's LiveIntervals) marks p and q as live across the same ranges derived above, confirming the hand-computed next-use points.
- GCC: `gcc -fdump-rtl-life` exposes RTL-level liveness (life_analysis) whose live-in/live-out sets for each insn match the manual bottom-up scan performed here.

Q6. RISC Target Code Generation for $(a+b)*(c-d)+e$

Target ISA: LOAD, STORE, ADD, SUB, MUL, MOV (RISC, register-register form, 3 general-purpose registers R0-R2).

Decomposition into three-address form

```
t1 = a+b
t2 = c-d
t3 = t1*t2
t4 = t3+e
```

Generated target instruction sequence

#	Instruction	Effect
1	LOAD R0, a	R0 = a
2	LOAD R1, b	R1 = b
3	ADD R0, R0, R1	R0 = a+b (t1)
4	LOAD R1, c	R1 = c
5	LOAD R2, d	R2 = d
6	SUB R1, R1, R2	R1 = c-d (t2)
7	MUL R0, R0, R1	R0 = (a+b)*(c-d) (t3)
8	LOAD R1, e	R1 = e
9	ADD R0, R0, R1	R0 = (a+b)*(c-d)+e (t4, final result)
10	STORE result, R0	Write result back to memory

The sequence uses only 3 registers at any time (never more than a, b or c, d live simultaneously alongside the running partial product), and needs no spill code — a direct benefit of evaluating the expression tree left-to-right so each sub-result is consumed as soon as it is produced.

Runtime storage management and stack frame layout

For a routine that computes this expression, a standard stack-based runtime layout is used:

High addresses

+-----+ Caller's frame +-----+	
+-----+ Return address +-----+	<- pushed by CALL
+-----+ Saved dynamic link (old FP) +-----+	<- Frame Pointer (FP)
+-----+ Saved callee-save registers +-----+	
+-----+ Local variables a, b, c, d, e, result +-----+	
+-----+ Temporaries / spill slots (t1..t4 if spilled) +-----+	<- Stack Pointer (SP)

Low addresses

- Activation record: created by the CALL sequence (push return address, save old frame pointer, allocate space for locals/temporaries) and torn down by RETURN (deallocate, restore FP, pop return address).
- Since all 4 temporaries (t1-t4) fit in the 3 available registers without spilling, the temporaries section of the frame is unused for this particular expression; it exists in the layout to hold register spills for larger expressions.
- Local variables a, b, c, d, e and result are addressed relative to FP (e.g. FP-4, FP-8, ...), giving position-independent access regardless of stack depth.

Tool verification

- GCC: gcc -march=riscv64 -S on the equivalent C expression emits an lw/sub/add/mul/sw sequence and a prologue/epilogue matching this frame layout.
- LLVM: llc -march=riscv64 on the IR produces an analogous RV64 instruction sequence with the same register-reuse pattern.
- Spike / RARS: single-stepping the assembled RISC-V program in the Spike ISA simulator or RARS shows the register file updating exactly as in the 10-instruction table above.

Code of Conduct

I, Sudharsan D (192524063), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Sudharsa

